



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-20

今日榜单

- 1 affaan-m/ECC** JavaScript NEW
The agent harness performance optimization system. Skills, instincts, memory, se...
- 2 BuilderIO/agent-native** TypeScript NEW
A framework for building agentic apps
- 3 cloudflare/security-audit-skill** JavaScript 5天
A coding-agent skill for multi-phase security audits with independently verified...
- 4 trycua/cua** HTML 2天
Scale computer-use 2.0 with open-source drivers, cross-OS fleets, and benchmarks...
- 5 anthropics/financial-services** Python NEW
- 6 paperless-ngx/paperless-ngx** Python NEW
A community-supported supercharged document management system: scan, index and a...
- 7 anthropics/claude-code** TypeScript 4天
Claude Code is an agentic coding tool that lives in your terminal, understands y...
- 8 mihail911/modern-software-dev-assignments** Python NEW
Assignments for CS146S: The Modern Software Dev (Stanford University Fall 2026/2...
- 9 higgsfield-ai/higgsfield** Jupyter Notebook 2天
Fault-tolerant, highly scalable GPU orchestration, and a machine learning frame...
- 10 Open-Dev-Society/OpenStock** TypeScript 2天
OpenStock is an open-source alternative to expensive market platforms. Track rea...

#1

JavaScript

NEW

ECC

The agent harness performance optimization system. Skills, instincts, memory, security, and research-first development for Claude Code, Codex, Opencode, Cursor and beyond.

Stars **263,339** Forks **39,404** Today **+1,012**

<https://github.com/affaan-m/ECC>

项目简介：ECC 是一个面向 AI 编码代理 (Agent) 的"性能优化操作系统", 为 Claude Code、Codex、Cursor、Opencode 等主流 AI 编程工具提供统一的能力增强层。它试图解决当前 AI Agent 普遍存在的"记忆缺失、技能零散、安全无保障、缺乏研究习惯"等问题, 把零散的提示词技巧沉淀为一套可复用的工程化系统。

核心功能：一是 **Skills (技能库)**, 把常见的开发任务封装成可调用的能力模块; 二是 **Instincts (本能)**, 让 Agent 在特定情境下自动触发行为, 减少人工提示; 三是 **Memory (记忆)**, 跨会话保存上下文与偏好; 四是 **Security (安全)**, 通过 `ecc-agentshield` 等包防护代理行为; 五是 **Research-first 开发范式**, 引导 Agent 先调研再动手, 降低幻觉风险。

适用场景：主要面向重度使用 AI 编程代理的开发者、团队和 AI 工具链集成者。典型场景包括: 让 Claude Code 记住项目规范并自动遵守、为多代理协作提供统一的能力底座、以及在 CI 或本地环境中对 Agent 行为做安全审计。对个人开发者而言, 它更像是"给 AI 助手装上一套外挂系统"。

技术亮点：项目以 JavaScript/TypeScript 为主, 同时兼容 Shell、Python、Go、Java、Perl 等多种语言生态, 说明其设计目标是"语言无关的 Agent 协议层"。通过 npm 包 (`ecc-universal`、`ecc-agentshield`)、GitHub App、Claude Code 插件 (`ecc@ecc`) 多渠道分发, 并特别强调"仅从官方渠道安装"以防恶意镜像——这反映出 Agent 生态中供应链安全的现实焦虑。

上手建议：推荐从官方 GitHub 仓库或 `ecc.tools` 网站开始, 按 README 的"guided setup"完成 Claude Code 插件安装, 避免手动叠加多种安装方式造成冲突。想深入的话可先读 docs/ 下的多语言文档和 `ecc-agentshield` 源码, 理解安全层设计; 贡献方面建议先加入 Discord 社区, 从文档翻译 (项目已支持 13 种语言) 或 Skill 模板扩充这类低门槛任务切入。

注: 263k 的星标数对于单个项目而言异常之高, 建议核实数据来源是否为聚合统计或包含镜像仓库, 实际影响力可能需以 npm 下载量和 Discord 活跃度为准。

#2

TypeScript

NEW

agent-native

A framework for building agentic apps

Stars **5,010** Forks **470** Today **+89**

<https://github.com/BuilderIO/agent-native>

项目简介：Agent-Native 是 BuilderIO 推出的开源 TypeScript 框架，用于构建"带界面的 AI Agent 应用"。它要解决的核心问题是：当前大多数 Agent 只能通过聊天框交互，而真实的知识工作（如文档编辑、设计、会议记录）需要可视化界面来查看、审批和协作。它让 Agent 和 UI 共享同一套能力与数据，而非让 Agent 去"模拟点击"界面。

核心功能：

- **共享 Action 层：**每个能力只需定义一次（基于 Zod schema），Agent 将其作为工具调用，UI 通过 useActionQuery 调用，同时自动暴露为 HTTP、MCP、A2A 和 CLI 接口。
- **共享数据与状态：**Agent 的产出实时出现在 UI 中，UI 的操作对 Agent 可见，Agent 还能感知当前页面、选中记录等 UI 上下文。
- **内置企业级能力：**Agent 聊天、认证与权限、技能与记忆、自动化调度、Agent 团队协作、PostgreSQL/PGLite 数据库后端一应俱全。
- **开箱即用的示例 Agent：**提供 Clips（会议/屏幕录制理解）、Design 等开源 Agent 作为起点。

适用场景：适合需要"人机协作"而非纯自动化的知识工作类产品，比如会议纪要工具、设计协作平台、内容审核系统等。目标用户是希望快速搭建 Agent 应用的全栈 TypeScript 开发者，以及需要为现有产品嵌入 Agent 能力的团队。

技术亮点：最巧妙的设计是"Action 即一切"——单个 defineAction 同时驱动 UI、Agent、HTTP、MCP、A2A 和 CLI 六种入口，避免了传统方案中为 Agent 和 UI 分别维护两套逻辑的重复。此外，Agent 不通过点击 UI 操作，而是走与 UI 相同的 action 层，保证了权限校验和行为一致性；使用 PGLite 支持本地开发、PostgreSQL 用于生产，降低了上手门槛。

上手建议：最快的方式是运行 `npx @agent-native/core@latest create my-agent --standalone --template chat` 创建模板项目，然后阅读官方 getting-started 文档，重点理解 defineAction 的写法。想深入的话，建议先研究仓库中 Clips 示例，看它如何把录制、转录、总结等能力组织成共享 action；贡献方面可从补充示例 Agent 或完善文档入手。

#3

JavaScript

连续 5 天

security-audit-skill

A coding-agent skill for multi-phase security audits with independently verified, machine-readable findings

Stars **17,510** Forks **971** Today **+2,375**

<https://github.com/cloudflare/security-audit-skill>

项目简介：这是 Cloudflare 开源的一个"编码智能体技能包"（coding-agent skill），它把通用的 AI 编码助手（如 Claude Code 等）转化为一个结构化的安全审计员。它解决的核心问题是：让 AI 做安全审计时不再"拍脑袋下结论"，而是通过多阶段流程产出可独立验证、机器可读的漏洞发现。该项目源自 Cloudflare 内部的漏洞发现框架（vulnerability harness），是其单仓库起点版本。

核心功能：整个审计流程被拆成六个阶段——侦察（绘制架构与信任边界）、覆盖驱动的猎取（按覆盖账本分配独立猎手）、候选验证（每个候选交给全新验证者尝试证伪）、结构化输出（写入 findings.json 并按 schema 校验）、独立记录复核（新智能体复核最终结论）、目标中立报告（生成 REPORT.md 等文档）。它用三个明确裁决区分结果：confirmed（有完整来源追踪）、needs_validation（有确切未决事实但不给严重级别）、rejected（已证伪）。此外，多次运行是累加式的，会利用历史账本定位缺口、重验已变更代码，而不把陈旧或未决工作当作已覆盖。

适用场景：主要面向安全工程师、渗透测试人员，以及希望用 AI 批量审计代码库的团队。典型场景包括：对自研代码库做系统性安全审查、在 CI/CD 或发布流程前发现漏洞、针对特定目标类型（如 LLM 应用、Web 协议、供应链、云部署、客户端等）做专项猎取。由于内置了针对不同目标的攻击类别提示（内存安全、AI/LLM 提示注入、供应链、数据隔离等），它也适合作为安全团队搭建自有漏洞挖掘框架的起点。

技术亮点：最值得关注的是它的"对抗式验证"设计——每个候选漏洞都必须经过一个独立的新智能体尝试证伪，最终结论还要再经一轮独立复核，这大幅降低了 AI 幻觉导致的误报。其次是"覆盖账本"机制，用 coverage-ledger.json 追踪哪些单元已被检查、哪些存在缺口，避免审计盲区。工程上还提供了零依赖的校验脚本（validate-findings.cjs、validate-coverage-ledger.cjs）和 JSON schema，保证输出机器可读且可被下游工具消费，同时用测试文件保证生产者兼容性。

上手建议：最直接的方式是用 Skills CLI 安装：`npx skills add https://github.com/cloudflare/security-audit-skill --skill security-audit`，然后在目标代码库中启动编码智能体并说"security audit this codebase"。建议先从阅读 SKILL.md 了解核心原则和工作流概

览，再按需查阅各攻击类别文档（如 `AI-AND-LLM.md`）。若想贡献，可从 `validate-*.test.cjs` 的测试用例入手理解数据格式约定，或针对尚未覆盖的攻击类别补充提示词；同时推荐先读 Cloudflare 官方博客《Build your own vulnerability harness》理解设计背景。

#4

HTML

连续 2 天

cua

Scale computer-use 2.0 with open-source drivers, cross-OS fleets, and benchmarks for training, evaluation, and data generation.

Stars **24,915** Forks **1,711** Today **+1,012**

<https://github.com/trycua/cua>

项目简介： Cua (trycua/cua) 是一个为 AI Agent 提供“可操作计算机”能力的开源基础设施项目。它通过桌面自动化驱动、隔离云桌面、本地 macOS 虚拟机和评测基准，让 AI 能像人一样操作图形界面完成真实任务，解决的是 Computer-Use Agent 从“能看”到“能动手做”的落地问题。

核心功能：

- **Cua Fleets：** 按需分配隔离的云桌面沙箱，支持运行命令、截图和与应用程序交互，实现跨操作系统的大规模 Agent 部署。
- **Cua Driver：** 跨 macOS、Windows、Linux 的桌面自动化驱动，可检查和操作原生应用（如 LibreOffice、Inkscape）。
- **Lume：** 在 Apple Silicon 上本地运行 macOS/Linux 虚拟机，提供轻量级本地测试环境。
- **CUA-S1：** 面向计算机使用决策的小型专用模型，可配合任意 Agent 框架使用。
- **Cua Bench：** 创建任务、评估 Agent 表现并导出轨迹数据，用于训练和基准测试。

适用场景：

- AI 研究者与工程师训练、评测 Computer-Use Agent，需要标准化任务环境和轨迹数据。
- 开发者构建需要操作 GUI 的自动化流程（如 RPA 升级版），在云或本地沙箱中安全运行。
- 企业需要跨操作系统批量部署 Agent 执行重复性桌面任务，同时保持环境隔离和可复现。

技术亮点：

- 提出“Computer-Use 2.0”概念，强调 Agent 在同一任务中灵活切换代码、API 和图形界面，而非局限于单一交互模式。
- 提供从本地 VM (Lume) 到云 Fleets 的统一抽象，兼顾开发调试与规模化部署。
- 自带基准测试和轨迹导出能力，形成“训练—评测—数据生成”的闭环，降低 Agent 研发门槛。

上手建议： 从官方文档的教程入手：先尝试“Cua Driver 操作计算器”或“Lume 创建 Tahoe VM”感受本地能力，再通过 run.cua.ai 体验 Fleets 云桌面。想贡献代码可关注 libs/cua-s1 模型库和 Cua Bench 任务定义，Discord 社区是获取支持的好渠道。

#5

Python

NEW

financial-services

Stars **35,220** Forks **5,236** Today **+236**<https://github.com/anthropics/financial-services>

项目简介：这是 Anthropic 官方开源的金融服务业 AI 智能体参考实现库，提供投资银行、股票研究、私募股权和财富管理等场景的现成 Agent、技能（Skills）与数据连接器。它解决的核心问题是：让金融机构不必从零搭建 AI workflow，而是基于经过设计的参考模板快速落地，同时通过"人类签字确认"的设计守住合规底线。

核心功能： - **九大命名 Agent：**覆盖 Pitch Agent（路演材料）、Market Researcher（行业研究）、Earnings Reviewer（财报点评）、Model Builder（DCF/LBO/三表模型）、GL Reconciler（总账对账）、KYC Screener（开户审查）等端到端 workflow。 - **垂直技能插件：**可按需单独安装 /comps、/dcf、/earnings 等斜杠命令与数据连接器，无需部署完整 Agent。 - **一套源码双路部署：**同一套系统提示词和技能，既能作为 Claude Cowork 插件安装，也能通过 Claude Managed Agents API 部署到自有 workflow 引擎中。 - **合作伙伴插件：**已集成 LSEG、S&P Global 等金融数据商的连接器。

适用场景：主要面向投行分析师、股票研究员、PE 投资人、基金行政与财务运营团队，用于制作 pitch deck、更新财务模型、撰写研究笔记、月末结账、LP 报表审计和 KYC 合规筛查等日常高频工作。所有输出都定位为"待人类复核的草稿"，而非最终决策。

技术亮点： - **合规优先的设计哲学：**README 明确声明不构成投资建议，Agent 不执行交易、不入账、不批准开户，所有产出均需专业人员签字，这在金融 AI 项目中是关键信任基础。 - **双形态架构：**通过 managed-agent-cookbooks/ 提供 agent.yaml、子 Agent 和 steering-event 示例，把 Cowork 插件平滑映射到 Managed Agents API，避免重复建设。 - **自包含插件：**每个 Agent 打包其依赖的 skills，安装即用，降低配置摩擦。

上手建议：若只想试用，可在 Claude Code 中先 `claude plugin marketplace add anthropics/financial-services`，再安装 financial-analysis 核心技能，然后按需挑选单个 Agent（如 pitch-agent）；若要自建 workflow，直接参考 managed-agent-cookbooks/ 中对应目录的 agent.yaml。贡献方向可关注 partner-built/ 连接器扩展或新增垂直场景的 skills 包。

#6

Python

NEW

paperless-ngx

A community-supported supercharged document management system: scan, index and archive all your documents

Stars **45,388** Forks **3,137** Today **+32**

<https://github.com/paperless-ngx/paperless-ngx>

项目简介：Paperless-ngx 是一个社区驱动的开源文档管理系统，目标是把纸质文件变成可搜索的在线档案库，真正实现“无纸化”。它是原 Paperless 和 Paperless-ng 项目的官方继任者，由团队共同维护而非个人主导，目前已获得超过 4.5 万星标，社区活跃度极高。

核心功能：一是文档扫描与 OCR 识别，将纸质文件转为可全文检索的数字档案；二是自动索引与标签分类，支持基于内容自动匹配标签、correspondent 和文档类型；三是全文搜索，能快速定位任意文档中的关键词；四是多格式支持与归档，兼容 PDF、图片等多种格式并统一管理；五是提供 Web 界面和 REST API，方便集成与远程访问。

适用场景：适合需要管理大量纸质或电子文档的个人用户、家庭办公室、小型企业以及注重数据隐私的自托管爱好者。典型场景包括：将发票、合同、收据扫描归档后随时检索，替代传统文件夹和网盘的手动整理方式，以及通过 Docker 自建私有文档云，避免敏感资料上传到第三方服务。

技术亮点：项目采用 Python 编写，基于 Django 框架构建后端，结合 OCR（如 Tesseract）实现文档内容提取，并使用 Whoosh 或类似引擎提供全文检索。部署上以 Docker Compose 为核心，大幅降低自托管门槛，同时提供官方演示站点和完整的 REST API，架构清晰且易于扩展。

上手建议：最快的方式是运行官方提供的一键安装脚本，通过 Docker Compose 拉起完整环境，然后访问 Web 界面导入文档即可。如果想参与贡献，可以从修复 bug、改进翻译（项目使用 Crowdin 做多语言）或完善文档入手，社区通过 Matrix 聊天室提供支持，适合新手逐步融入。

#7

TypeScript

连续 4 天

claude-code

Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands.

Stars **146,929** Forks **23,989** Today **+415**

<https://github.com/anthropics/claude-code>

项目简介： Claude Code 是 Anthropic 官方推出的终端智能体（Agentic）编程工具，把 Claude 大模型直接嵌入开发者的命令行环境，让 AI 能"读懂"整个代码库并用自然语言指令完成编码任务。它解决的核心问题是：把 AI 编程助手从"补全单行代码"升级为"理解项目全局、自主执行多步任务"的智能体。

核心功能： 1. **自然语言驱动的任务执行：** 用日常语言下达指令，自动完成重构、调试、写测试等常规开发任务。 2. **代码库理解与解释：** 能通读整个项目结构，解释复杂代码逻辑，回答"这段代码在做什么"。 3. **Git 工作流自动化：** 自动处理提交、分支、PR 等版本控制操作，减少手工操作。 4. **多入口接入：** 支持终端、IDE 插件，以及在 GitHub 上通过 @claude 标签触发，覆盖不同工作习惯。 5. **插件扩展机制：** 仓库内置插件目录，可通过自定义命令和 Agent 扩展能力边界。

适用场景： 主要面向需要频繁与代码打交道的开发者、技术团队和 DevOps 工程师。典型场景包括：接手陌生代码库时快速理解架构、批量执行重复性重构、在终端里不切换窗口完成"改代码—跑测试—提交"的闭环，以及把 AI 助手接入 CI/CD 或 GitHub 协作流程。

技术亮点： - 采用 **Agentic 架构**而非简单问答，能自主规划多步骤任务并调用工具（读写文件、执行命令、操作 Git）。 - 用 **TypeScript 编写并发布为 npm 包**，跨平台分发友好；同时提供 curl、Homebrew、WinGet 等多种安装方式，说明其在安装体验上做了工程化考量（npm 已被标记为 deprecated，转向更原生的安装渠道）。 - 提供**明确的数据使用与隐私策略**，强调会话数据有限保留、不用于模型训练，这对企业级采用是关键卖点。

上手建议： 最快路径是按 README 推荐方式安装（Mac/Linux 用 curl 脚本或 Homebrew，Windows 用 PowerShell 或 WinGet），然后 cd 到项目目录运行 claude 即可。想深入可先读官方文档（code.claude.com/docs），了解插件机制后再看仓库的 plugins/ 目录；遇到问题用 /bug 命令或提 GitHub Issue，也可加入官方 Discord 社区交流。

#8

Python

NEW

modern-software-dev-assignments

Assignments for CS146S: The Modern Software Dev (Stanford University Fall 2026/2025)

Stars **4,438** Forks **1,006** Today **+174**

<https://github.com/mihail911/modern-software-dev-assignments>

项目简介

这是斯坦福大学 CS146S《现代软件开发者》课程的官方作业仓库，配合课程网站 themodernsoftware.dev 使用。它并非一个可直接运行的工具，而是一套结构化的编程练习，帮助学习者系统掌握现代软件开发的核心工作流与工程实践。

核心功能

- 提供多个分模块的编程作业（assignments），覆盖从环境搭建到完整项目开发的流程。
- 附带完整的 Python 3.12 开发环境配置指南（Conda + Poetry）。
- 通过真实可运行的代码骨架，引导学习者实践依赖管理、测试、CI/CD 等现代开发环节。
- 与斯坦福课程内容同步，作业设计贴合课堂教学进度。

适用场景

- 正在学习或自学 CS146S 课程的学生，需要完成并提交作业。
- 想系统补全“现代软件工程”技能（如 Poetry、Conda、自动化测试）的 Python 开发者。
- 教师或助教参考作业设计，用于自己的课程建设。

技术亮点

- 采用 Conda 管理 Python 版本、Poetry 管理项目依赖，清晰分离环境与包管理，符合现代 Python 工程最佳实践。
- 锁定 Python 3.12，避免因版本差异导致作业运行问题。
- 仓库结构简洁，每个作业独立成模块，便于按需学习和测试。

上手建议

1. 先阅读 README，严格按步骤安装 Anaconda、创建 cs146s 环境、安装 Poetry 并执行 `poetry install`。
2. 从第一个作业目录开始，阅读其中的说明文件或代码注释，运行测试或示例以验证环境。
3. 若想贡献，可关注 issue 区或向课程团队反馈作业中的问题；但注意这是课程作业仓库，通常不接受外部功能 PR，更适合作为学习参考。

#9

Jupyter Notebook

连续 2 天

higgsfield

Fault-tolerant, highly scalable GPU orchestration, and a machine learning framework designed for training models with billions to trillions of parameters

Stars **5,174** Forks **933** Today **+461**

<https://github.com/higgsfield-ai/higgsfield>

项目简介：Higgsfield 是一个开源的多节点 GPU 编排与机器学习框架，目标是让开发者能够轻松训练从十亿到万亿参数级别的大模型（如 LLM），而无需为分布式训练的复杂性和环境配置而头疼。它同时充当 GPU 资源管理器和训练框架，把“多机训练”这件事从一件令人崩溃的苦差事变成一条顺畅的工作流。

核心功能： 1. **GPU 资源编排：**为用户分配独占或非独占的计算节点，并通过队列机制管理多个实验之间的资源竞争。 2. **分布式训练支持：**兼容 DeepSpeed ZeRO-3 和 PyTorch FSDP API，支持万亿参数模型的高效分片训练。 3. **实验生命周期管理：**提供训练任务的启动、执行和监控框架，并自动保存 checkpoint。 4. **GitHub CI/CD 集成：**通过与 GitHub 和 GitHub Actions 的无缝对接，实现机器学习开发的持续集成与自动部署。 5. **极简实验 API：**通过 @experiment 装饰器等简单接口定义训练流程，无需编写大量配置参数或 YAML 文件。

适用场景：主要面向需要训练大规模语言模型的研究团队和工程团队，尤其是那些拥有多台 GPU 服务器（如 Azure、LambdaLabs、FluidStack 等云平台上的节点）但缺乏成熟编排工具的团队。典型场景包括：在自有节点集群上分布式训练 LLaMa 等大模型、需要管理多个实验排队运行、以及希望通过 GitHub 工作流自动化模型训练与部署。

技术亮点：最值得关注的是它“遵循标准 PyTorch 工作流”的设计哲学——不发明新的训练范式，而是让用户继续使用熟悉的 PyTorch/DeepSpeed/Accelerate 代码，只在其上叠加编排层。此外，它通过 GitHub Actions 驱动节点上的代码部署和实验运行，把 MLOps 的 CI/CD 理念真正落地到了分布式训练场景中，同时解决了“环境地狱”和“配置地狱”两个长期痛点。

上手建议：建议从 README 中链接的 setup.md 快速开始指南入手，依次完成项目初始化、环境配置、Git 设置和节点接入，然后运行第一个实验。需要注意的是，当前项目需要用户自备 Ubuntu 节点并具备 SSH 和非 root sudo 权限，且版本号仍为 0.0.3，属于早期阶段，适合愿意尝鲜并反馈问题的开发者。若想贡献代码，可以先从兼容更多云平台或完善文档入手。

#10

TypeScript

连续 2 天

OpenStock

OpenStock is an open-source alternative to expensive market platforms. Track real-time prices, set personalized alerts, and explore detailed company insights — built openly, for everyone, forever free.

Stars **16,498** Forks **2,118** Today **+752**

<https://github.com/Open-Dev-Society/OpenStock>

项目简介：OpenStock 是一个开源的股票市场追踪平台，定位为昂贵商业行情软件（如各类付费终端）的免费替代品。它让普通用户无需订阅费即可查看实时股价、设置提醒并了解公司基本面信息，所有代码公开透明，永久免费。

核心功能：实时股价追踪与行情展示；个性化价格提醒（通过邮件触发）；公司详情与洞察（借助 TradingView 图表和 Finnhub 数据）；基于 Better Auth 的用户账户体系；自动化后台任务调度（定时拉取行情、发送通知）。

适用场景：适合个人投资者、量化爱好者或学生用来自建一个轻量的行情看板；也适合开发者作为学习 Next.js 全栈 + 金融数据 API 集成的实战项目。由于 AGPL-3.0 协议，若二次部署为公开服务需开源修改代码。

技术亮点：采用 Next.js + TypeScript + Tailwind + shadcn/ui 的现代前端栈，UI 质量较高；用 Inngest 处理事件驱动的后台任务（如定时抓价、触发提醒），MongoDB 存储数据，Finnhub 提供行情源，TradingView 负责图表可视化——整体是一套典型的“Serverless 风格”全栈金融应用参考架构。

上手建议：先阅读 README 中的环境变量配置部分，准备好 Finnhub API Key、MongoDB 连接串和邮件服务凭据，然后本地 `npm install && npm run dev` 跑起来。想贡献的话，可从 issue 列表中的小功能或文档改进入手，注意遵守 AGPL-3.0 的署名与开源义务。

数据来源: github.com/trending

报告日期: 2026-09-20

由 DeepSeek AI 提供智能分析 | 自动生成